Chương 4: JS

TS Nguyễn Mạnh Tuấn
tuannm@ueh.edu.vn

Nội dung

- Giới thiệu JS
- Hành trình trở thành anh hùng JS
- DOM trong JS
- Event trong JS

Giới thiệu JS

JavaScript là gì?

- **JavaScript** là ngôn ngữ lập trình kịch bản dựa trên đối tượng, được hỗ trợ bởi hầu hết trình duyệt hiện đại và 99% website.
- **Điểm chính:**
  - **Linh hoạt:** Chạy đa nền tảng.
  - **Ứng dụng rộng:** Dùng cho web, game, mobile app và backend (Node.js).
  - **Hệ sinh thái mạnh:** Được các công ty lớn hỗ trợ với nhiều framework phổ biến như **ReactJS, Angular, Vue.js, Node.js, jQuery, KnockoutJS**.

JavaScript là gì?

- **Javascript** là ngôn ngữ kịch bản ở phía **client**.
- **Javascript** chịu ảnh hưởng của nhiều ngôn ngữ và có cú pháp gần giống với ngôn ngữ **Java**, nhưng dễ lập trình hơn.
- **Javascript** không có liên quan gì với ngôn ngữ **Java**.

JavaScript là gì?

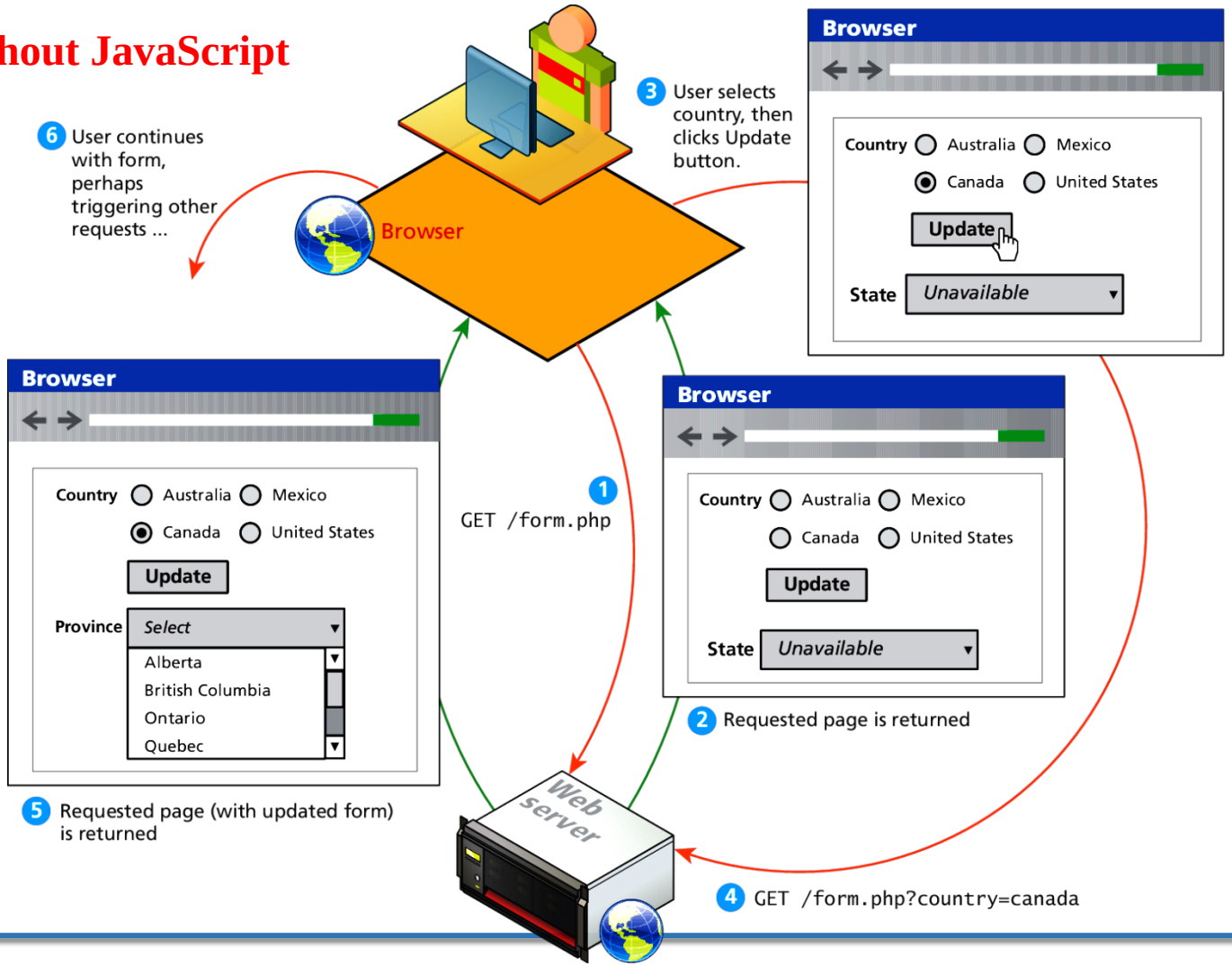
- Các ứng dụng phổ biến của **Javascript** bao gồm:
  - Tương tác với **HTML**, thay đổi nội dung và định dạng website.
  - Xử lý các hành động của người dùng như nhấn chuột, gõ phím.
  - Kiểm tra dữ liệu form trước khi gửi về server.
  - Tạo và truy xuất thông tin từ **cookie**.
  - Dùng làm ngôn ngữ lập trình phía server với các framework như **Node.js**.

Ưu điểm của Javascript

- Quá trình xử lý có thể được giảm tải từ máy chủ sang máy khách, do đó **giảm tải cho máy chủ**.
- Trình duyệt có thể phản hồi các sự kiện của người dùng nhanh hơn so với yêu cầu đến máy chủ từ xa, điều này giúp **cải thiện trải nghiệm người dùng**.
- **JavaScript** tương tác với **HTML** đã tải, tạo trải nghiệm giống phần mềm máy tính để bàn thay vì chỉ là HTML đơn giản.

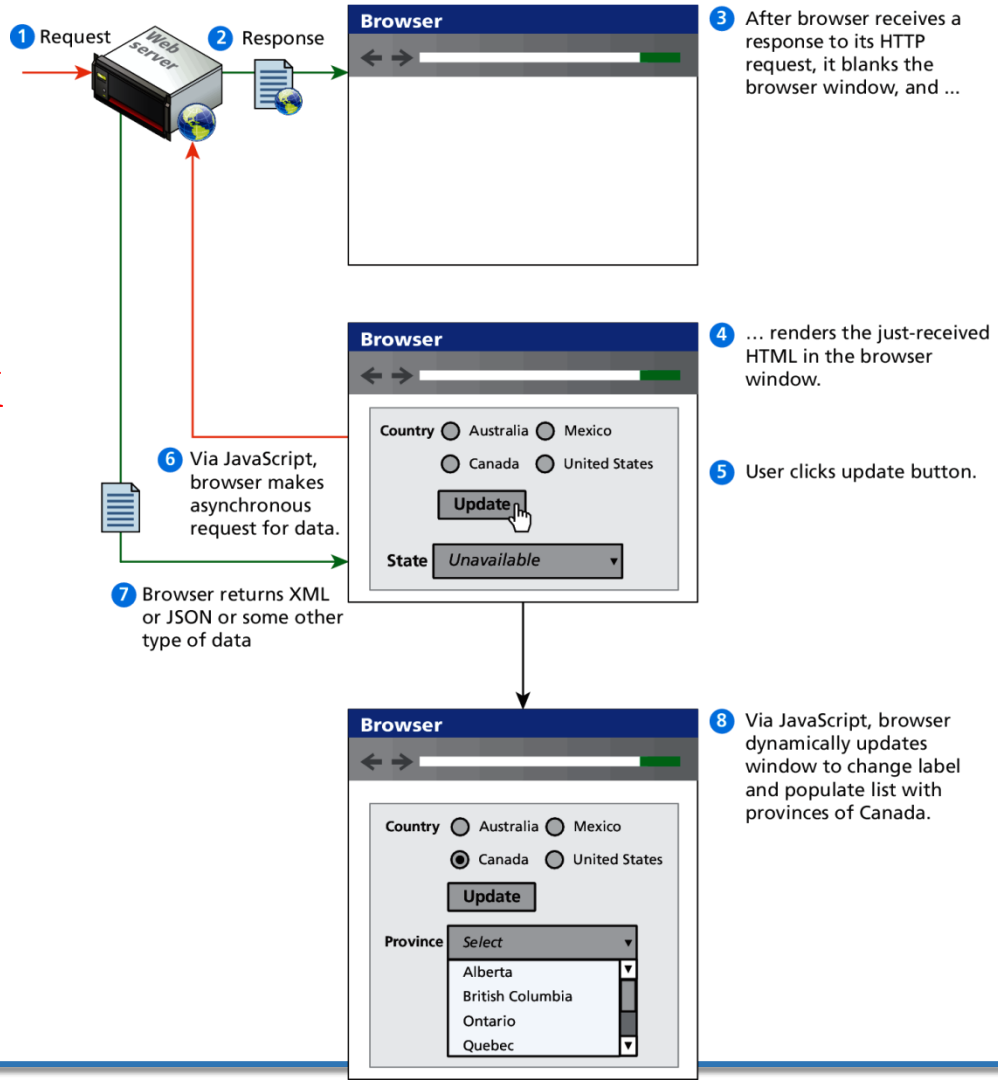
Asynchronous data requests

- **Without JavaScript**

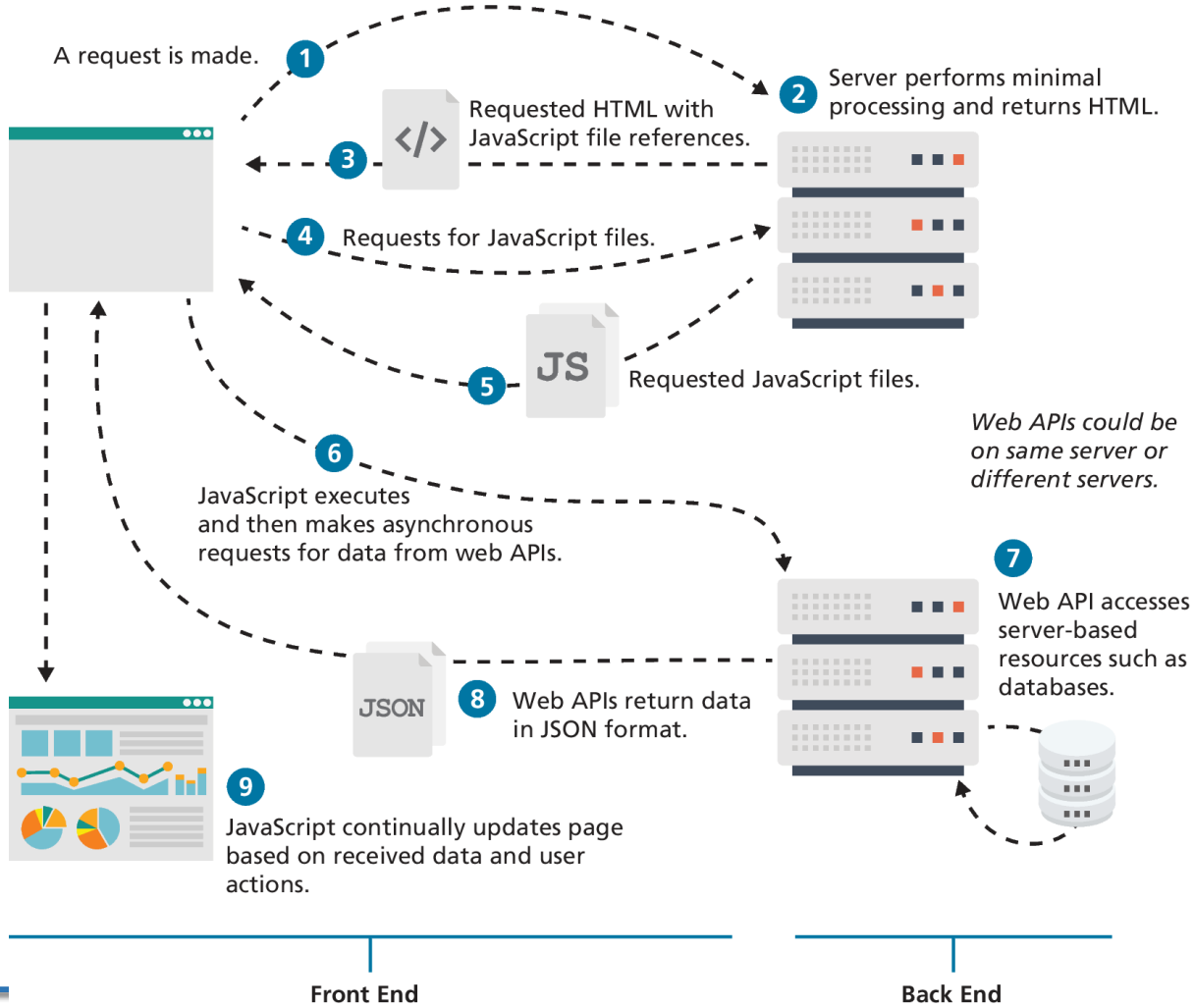


Asynchronous data requests

- **The better AJAX way**



JavaScript and Web 2.0



Sử dụng code Javascript trong trang HTML

- Chèn trực tiếp đoạn code Javascript vào giữa cặp thẻ `<script> … </script>` trong tập tin HTML.
- Sử dụng tập tin Javascript riêng (.js), và dùng thẻ `<script>` để kết nối với trang HTML.

```
<html lang="en">
<head>
  <title>JavaScript placement possibilities</title>
  <script>
    /* A JavaScript Comment */
    alert("This will appear before any content");
  </script>

  <script src="greeting.js"></script>
</head>
<body>
<h1>Page Title</h1>

<a href="JavaScript:OpenWindow();">for more info</a>
<input type="button" onClick="alert('Are you sure?');" />

<script>
   alert("Hello World");
</script>
```

Embedded JavaScript

External JavaScript

Inline JavaScript

Embedded JavaScript

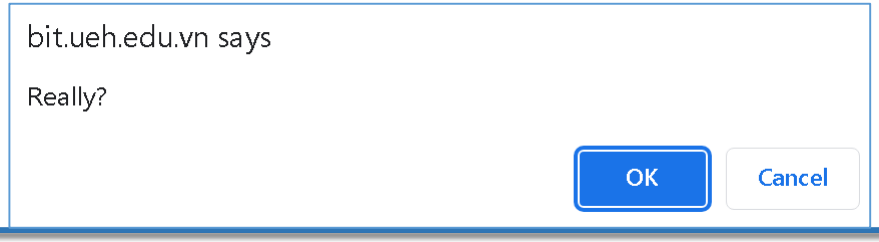
Sử dụng code Javascript trong trang HTML

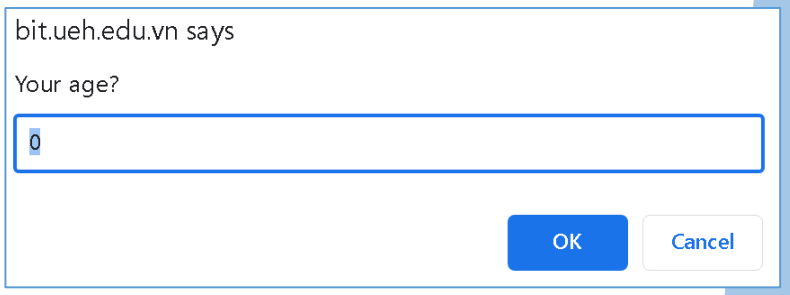
Một số lưu ý:

- Ta có thể chèn đoạn mã **Javascript** vào bất kỳ vị trí nào trong trang web, cả phần **<body>** và **<head>**.
- **Script** trong phần **<head>** thường là khai báo hàm (**function**).
- **Script** trong phần **<body>** tạo nội dung trang, thực thi khi nạp trang.
- Một tập tin **HTML** có thể chứa nhiều đoạn mã **Javascript**, không giới hạn số lượng.

Những hàm Output thường dùng

- console.log(a); // write to the browser console
- document.write(a); // write to the HTML
- alert(a); // output in an alert box
- confirm("Really?"); // yes/no dialog, returns true/false depending on user click
- prompt("Your age?","0"); // input dialog. Second argument is the initial value





Hành trình trở thành anh hùng JS

Học JS qua game: Anh Hùng Javascript

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Anh Hùng Javascript</title>
    <script type="text/javascript">
            // Gõ code ở đây
    </script>
    <script type="text/javascript" src="anhhung.js"></script>
</head>
<body>

</body>
</html>
```

Quest 1: Kiến tạo Anh Hùng – Biến và kiểu dữ liệu

- Việc lưu trữ dữ liệu là một trong những điều quan trọng nhất khi lập trình.
- Thông thường, các dữ liệu tạm sẽ được lưu giữ bằng các biến (variable) trong bộ nhớ.
- Đối với Javascript, bạn cần khai báo biến bằng từ khóa var trước khi gán dữ liệu cho biến đó

```
var age = 18;                                   // number
var name = "Jane";                              // string
var name = { first: "Jane", last: "Doe" };  // object
var truth = false;                              // boolean
var sheets = ["HTML", "CSS", "JS"];           // array
var a; typeof a;                                // undefined
var a = null;                                  // value null
```

Biến

- **String** dùng để chứa chuỗi kí tự và phải được bao quanh bởi cặp nháy đơn ('…') hay nháy đôi ("…").
- **Number** dùng để chứa dữ liệu kiểu số nguyên, số thập phân và không nằm trong cặp nháy đơn hay nháy đôi.
- **Boolean** là kiểu logic, chỉ có 2 giá trị là đúng (true) hoặc sai (false).
- **Object** là một đối tượng nói chung với các thuộc tính và phương thức riêng.
- **Array** là mảng, dùng để chứa tập hợp nhiều biến.

Khai báo biến

Đặc điểm	var	let	const	Biến toàn cục
Phạm vi	**Hàm** (function scope)	**Khối** (block scope)	**Khối** (block scope)	**Toàn cục** (global scope)
Có thể thay đổi giá trị	**Có thể thay đổi**	**Có thể thay đổi**	**Không thay đổi**	**Có thể thay đổi**
Hoisting	**Hoisted**, giá trị là undefined	**Hoisted**, nhưng không thể truy cập trước khởi tạo	**Hoisted**, không thể truy cập trước khởi tạo	**Hoisted**, thuộc tính của window (trình duyệt) hoặc global (Node.js)
Khai báo lại	**Có thể khai báo lại**	**Không thể khai báo lại**	**Không thể khai báo lại**	**Có thể khai báo lại**
Khai báo trong block	**Không** có block scope	**Có** block scope, an toàn hơn	**Có** block scope, không thay đổi	**Không** có phạm vi khối, dễ gây lỗi
Sử dụng phổ biến	**Tránh** trong ES6+, hỗ trợ trình duyệt cũ	**Thường dùng** khi cần thay đổi giá trị	**Thường dùng** khi không thay đổi giá trị	**Tránh** sử dụng, dễ gây lỗi

Bài tập 1

- Hãy kiếm 1 hình siêu anh hùng trên mạng để làm hình nền của file HTML trước.
- Tạo 1 biến chứa tên của bạn và MSSV
- Dùng JS để hiển thị chữ bên dưới trong HTML file:
  - Tôi tên là [Tên bạn] – MSSV: [MSSV] sẽ quyết tâm trở thành 1 anh hùng JS

Bài tập 2

- Khởi tạo thêm 3 biến để chứa thông tin về cấp độ (level), điểm kinh nghiệm (xp) và điểm kĩ năng (skillPoints). Giá trị khởi tạo của từng biến như sau:
  - Cấp độ: 1
  - Điểm kinh nghiệm: 0
  - Điểm kĩ năng: 5

Quest 2: Sức mạnh tính toán – Các toán tử cơ bản

- Javascript hỗ trợ đầy đủ các toán tử cơ bản cộng trừ nhân chia
- Thứ tự tính toán (trong trường hợp không sử dụng dấu ngoặc tròn để gom nhóm ưu tiên) là nhân chia trước, cộng trừ sau và từ trái sang phải.
  - `a = b + c - d;  // addition, substraction`
  - `a = b  (c / d);  // multiplication, division`
  - `x = 100 % 48;  // modulo. 100 / 48 remainder = 4`
  - `a++; b--;  // postfix increment and decrement`

Xem thêm tại link

Bài tập

- Sau khi hoàn thành các bài học trên thì anh hùng của bạn đã được nâng cấp:
  - level tăng thêm 1
  - Điểm kinh nghiệm: 150 * (level – 1) + 200
  - skillPoints: skillPoints + xp / 10
- Sau đó hãy cho in ra:
  - ten + " sẽ là Anh Hùng Javascript“
  - "Cấp độ hiện tại của bạn là " + level

Bài tập 3

- Khởi tạo thêm 3 biến là strength, agility và intel. Sử dụng giá trị của điểm kĩ năng (skillPoints) phía trên và tính toán giá trị khởi tạo của các biến:
  - strength bằng 60% điểm kĩ năng cộng với 10% của điểm kinh nghiệm.
  - agility bằng 20% điểm kĩ năng cộng với 20% của điểm kinh nghiệm.
  - intel bằng 20% điểm kĩ năng cộng với 1000% của cấp độ.
- Sau khi tính toán xong, hãy xuất các chỉ số theo cú pháp: "Chỉ số strength của bạn là " + chỉ số
- **LEVEL UP!** Chúc mừng bạn lên cấp 3! **XP +200**

Quest 3: Sức mạnh logic – Các toán tử so sánh

- Trên con đường hành hiệp, Anh Hùng sẽ gặp rất nhiều cám dỗ và lựa lọc. Chính vì thế, Anh Hùng cần phải so sánh chính xác và sử dụng sức mạnh logic để nhận ra chân tướng đúng/sai của mọi sự vật, sự việc.
- Để so sánh giữa các biến hay các biểu thức, bạn có thể sử dụng === cho so sánh bằng, !== cho so sánh không bằng, < và > cho so sánh hơn kém.

```
a != b          // not equal
a = b           // assignment
a == b          // equals
a != b          // unequal
```

```
a === b         // strict equal
a !== b         // strict unequal
a < b   a > b   // less and
greater than
a <= b  a >= b  // less or equal,
greater or eq
```

Quest 4: Lựa chọn định mệnh – Điều kiện và rẽ nhánh

```
if ((age >= 14) && (age < 19)) {        // logical condition
    status = "Eligible.";               // executed if condition is true
} else {                                // else block is optional
    status = "Not eligible.";           // executed if condition is false
}

switch (new Date().getDay()) {      // input is current day
case 6:                         // if (day == 6)
text = "Saturday";
break;
case 0:                         // if (day == 0)
text = "Sunday";
break;
default:                        // else...
text = "Whatever";
}
```

Bài tập 4

- Yêu cầu người dùng nhập lại tên:
  - Nếu đúng tên như ở trên thì:
    - Ghi log: Chính xác! Bạn có trí nhớ rất tốt! intel +5!
  - Nếu sai:
    - Ghi log: Tệ thật! Tên chính mình mà không nhớ sao?! intel -5!
- Sử dụng hàm prompt('Bạn muốn chọn phân cấp nào? a: Chiến Binh | b: Sát Thủ | c: Thợ Săn') để yêu cầu người dùng nhập vào 1 kí tự hoặc là a, hoặc b, hoặc c. Giá trị nhập vào sẽ được xử lý như sau:
  - 1. Nếu là kí tự a:
    - Cập nhật phân cấp Anh Hùng (biến phancap): Chiến Binh.
    - Điều chỉnh chỉ số: strength +15, agility -10, intel -5
  - 2. Nếu là kí tự b:
    - Cập nhật phân cấp Anh Hùng (biến phancap): Sát Thủ.
    - Điều chỉnh chỉ số: strength -15, agility +10, intel +5
  - 3. Nếu là kí tự c:
    - Cập nhật phân cấp Anh Hùng (biến phancap): Thợ Săn.
    - Điều chỉnh chỉ số: strength -15, intel +15
  - 4. Nếu không phải 3 trường hợp trên
    - Không cập nhật và điều chỉnh gì cả.

LEVEL UP! Chúc mừng bạn lên cấp 4! **XP +700**

Quest 5: Đấu trường Sinh Tử – Vòng lặp

- Anh Hùng nào cũng phải trải qua quá trình tập luyện và đấu tranh gian khổ để đạt đến vinh quang. Quá trình đó đòi hỏi những nỗ lực lặp đi lặp lại đến khi đạt được một ý đồ, một mục đích nào đó.

Chú thích

`j < 10`	Điều kiện duy trì vòng lặp. Dành cho cả FOR và WHILE.
`var j = 1`	Khai báo và khởi tạo các biến. Cần thiết cho vòng lặp FOR.
`j++`	Biểu thức chạy sau mỗi lượt chạy của vòng lặp. Cần thiết cho vòng lặp FOR.

```
var i = 1;
while ( i < 10 ) {
    // Thân vòng lặp
    console.log(i);
    i++;
}
// Vòng lặp kết thúc khi i = 10
```

```
for ( var j = 1 ; j < 10 ; j++ ) {
    // Thân vòng lặp
    console.log(j);
}
// Vòng lặp kết thúc khi j = 10
```

Quest 5: Đấu trường Sinh Tử – Vòng lặp

- **BATTLE!** Bạn chạm trán 1 con rồng già, và buộc phải giết nó để vượt qua
- Hàm Math.random(): dùng để tạo giá trị ngẫu nhiên từ 0 đến 1

```
/ ========================================================
 Trước khi vào trận chiến, cần xem lại các chỉ số của bạn
     và xem xét các thông tin của đối thủ.
 ========================================================/
level = 5; // Đảm bảo bạn sẽ ở cấp độ 5
var health = level  100;// Khi lượng máu <= 0 thì bạn sẽ chết
console.log(strength); // Quyết định mức sát thương
console.log(agility); // Quyết định xác suất né đòn
console.log(intel); // Quyết định xác suất chí mạng
/ ========================================================
 Rồng Già
 ——-
 Chậm chạp và yếu ớt hơn so với đồng loại, nhưng vẫn rất
 nguy hiểm với khả năng phun lửa có thể gây sát thương chí
 mạng rất cao
 ========================================================/
var dragonHealth = 600;
var dragonStr = 100;
var dragonAgi = 15;
var dragonInt = 80;
```

Bài tập 5

- Viết trò chơi Diệt Rồng theo các bước sau:
  - 1. Mỗi lượt chơi, Anh Hùng và Rồng lần lượt tấn công nhau.
    - => GỢI Ý: Sử dụng vòng lặp while.
  - 2. Anh Hùng tấn công trước, Rồng tấn công sau.
  - 3. Trò chơi kết thúc khi một phe hết máu (health < 0 hoặc dragonHealth < 0).
    - => GỢI Ý: đặt 1 biến boolean để kiểm tra.
  - 4. Sử dụng hàm (Math.random()*100) để lấy xác suất ngẫu nhiên theo %.
  - 5. Rồng né đòn thành công khi % xác suất ngẫu nhiên bé hơn dragonAgi.
    - => GỢI Ý: so sánh dragonAgi với (Math.random()*100).
  - 6. Anh Hùng né đòn thành công khi % xác suất ngẫu nhiên bé hơn agility.
    - => GỢI Ý: so sánh agility với (Math.random()*100) (khác ở 5).

Bài tập 5

- 7. Sát thương do Anh Hùng gây ra bằng với chỉ số strength.
  - => GỢI Ý: khi Anh Hùng đánh trúng, giảm máu Rồng theo giá trị sát thương.
- 8. Sát thương do Rồng gây ra bằng với chỉ số dragonStr.
  - => GỢI Ý: khi Rồng đánh trúng, giảm máu Anh Hùng theo giá trị sát thương.
- 9. Anh Hùng gây sát thương chí mạng khi % xác suất ngẫu nhiên lớn hơn intel.
  - => GỢI Ý: so sánh intel với 1 giá trị (Math.random()*100) (khác ở 5 và 6).
- 10. Rồng gây sát thương chí mạng khi % xác suất ngẫu nhiên lớn hơn dragonInt.
  - => GỢI Ý: so sánh dragonInt với 1 giá trị (Math.random()*100) (khác ở 5,6,9)).
- 11. Sát thương chí mạng sẽ gây thêm 200% sát thương.
  - => GỢI Ý: nếu gây sát thương chí mạng, giảm máu đi 2 lần sát thương.

BATTLE WON! Chúc mừng bạn đã đạt danh hiệu "Dũng sĩ diệt Rồng"!
LEVEL UP! Chúc mừng bạn lên cấp 6! **XP +600**

Code

```
var gameContinue = true;
while (gameContinue) {
    // Anh hùng tấn công!
    var heroHit = Math.random() * 100 > dragonAgi;
    if (heroHit) {
        dragonHealth = dragonHealth - strength;
        alert('TRẢM LONG! Bạn gây sát thương ' + strength + ' điểm cho Rồng!');
        var heroCritical = Math.random() * 100 < intel;
        if (heroCritical) {
            dragonHealth = dragonHealth - strength * 2; // Double damage
            alert('CHÍ MẠNG! Bạn gây sát thương chí mạng thêm ' + strength * 2 + ' điểm cho Rồng!');
        }
    } else alert('XÍ HỤT! Rồng đã lượn tránh được đòn tấn công của Bạn!');
    // Rồng tấn công!
    var dragonHit = Math.random() * 100 > agility;
    if (dragonHealth <= 0) {
        // Rồng lên bàn thờ
        gameContinue = false;
        alert('CHIẾN THẮNG!!! Chúc mừng bạn đã chiến thắng!!!');
    } else {
        if (dragonHit) {
            health = health - dragonStr;
            alert('HỎA CẦU! Rồng gây sát thương ' + dragonStr + ' điểm cho Bạn!');
            var dragonCritical = Math.random() * 100 < dragonInt;
            if (dragonCritical) {
                health = health - dragonStr * 2; // Double damage
                alert('CHÍ MẠNG! Rồng gây sát thương chí mạng thêm ' + dragonStr * 2 + ' điểm cho Bạn!');
            }
        } else alert('XÍ HỤT! Bạn đã tránh được đòn tấn công từ Rồng!');
    }

    // Kiểm tra xem Anh Hùng có bị nướng?!
    if (health <= 0) {
        gameContinue = false;
        alert('THẤT BẠI!!! Chúc mừng bạn đã có tên trên menu món tối của Rồng: Hiệp sĩ nướng lá lốt!');
    }
};
```

Bài tập 6 – Về nhà (cộng 2 lần phát biểu)

- Tự thiết kế giao diện trận đấu giữa anh hùng với rồng. Phải có các thông tin sau:
  - Hình người, Hình rồng, có thể kết hợp chuyển động mỗi khi rồng hay người ra chiêu hay bị đánh trúng, né được đòn.
  - Các thông tin khác như:
    - Lượng máu của người và rồng
    - Lịch sử các đòn đánh và né
    - ...

Quest 6: Tạo và gọi Hàm

- Hàm là 1 đoạn code riêng biệt có thể sử dụng nhiều lần. Hàm có thể nhận **tham số** từ biến và **trả kết quả** về để gán cho biến.
- Trước khi sử dụng, hàm cần được khởi tạo với từ khóa function tênHàm(thamSố1, thamSố2)
- Để bắt hàm trả về 1 giá trị nào đó, bạn cần sử dụng từ khóa return tênBiếnMuốnTrảVề;. return sẽ kết thúc hàm ngay lập tức, nên nếu có câu lệnh nào phía sau, câu lệnh đó sẽ không được thực thi.

Ví dụ

```
// Tạo hàm gây sát thương để giảm máu
// Tham số truyền vào: biến Number satThuong
// Kết quả trả về: biến Number luongMauHienTai
function gaySatThuong(luongMauHienTai, satThuong) {
    luongMauHienTai = luongMauHienTai – satThuong;
    return luongMauHienTai;
}
// Để sử dụng hàm, bạn cần phải gọi hàm
var luongMau = 100;
var satThuong = 49;
var luongMauConLai = gaySatThuong(luongMau, satThuong);
console.log(luongMauConLai);
```

Quest 6: Tạo và gọi Hàm

Traditional Syntax	Arrow Syntax	

`function () {`
`  statements`
`}`
	`() => {`
	`  statements`
	`}`
		Multi-line function, no parameters: {}, () required

`function (a,b) {`
`  statements`
`}`
	`(a,b) => {`
	`  statements`
	`}`
		Multi-line function, multiple parameters: () required

`function () {`
`  doSomething();`
`}`
	`() => {`
	`  doSomething();`
	`}`
		Single-line function, no return: {} required

`function (a) {`
`  return value;`
`}`
	`(a) => return value`	Single-line function, with return: {} optional

`function (a) {`
`  return value;`
`}`
	`a => value`	Single-line function, with return + one parameter: {}, (), return optional

`function () {`
`  return value;`
`}`
	`() => value`	Single-line function, with return + no parameters: {}, return optional () required

`function (a,b) {`
`  return value;`
`}`
	`(a,b) => value`	Single-line function, with return + multiple parameters: {}, return optional () required

`const g = function(a) {`
`  return value;`
`}`
	`const g = a => value`	Function expression

`function (a,b) {`
`  return {`
`  p1: a,`
`  p2: b`
`  }`
`}`
	`(a,b) => ({`
	`  p1: a,`
	`  p2: b`
	`  })`
		When arrow function returns an object literal, the object literal must be wrapped in parentheses.

Bài tập 7

- Viết lại trò chơi Diệt Rồng, có sử dụng hàm để:
  - 1. Tính toán khả năng né đòn.
  - 2. Tính toán lượng máu còn lại sau khi bị sát thương.
- **LEVEL UP!** Chúc mừng bạn lên cấp 7! **XP +700**

Quest 7: Đối tượng

- Đối tượng **Javascript** giống như một vật thể thực ngoài đời, có **tính chất (thuộc tính)** và **khả năng (phương thức/hàm)**.
- Hiểu theo cách khác, đối tượng trong **Javascript** là tập hợp của các **thuộc tính** và **phương thức** (cũng là hàm) bên trong.
- Có thể xem đối tượng như một **biến đặc biệt** trong **Javascript**, được khai báo trong cặp ngoặc nhọn {...} và sử dụng dấu . để truy xuất các **thuộc tính** và **phương thức** bên trong.

Ví dụ

```
// Cú pháp khai báo 1 đối tượng
var RongGia = {
    // Khai báo các thuộc tính
    name: 'Rồng Già',
    phanLop: 'Rồng',
    age: 6969,
    health: 600,
    strength: 100,
    agility: 15,
    intel: 80,
    // Khai báo các phương thức
    bay: function () {
        console.log('Flappy Dragon!');
    },
    phunLửa: function () {
        console.log('Rồng phun lửa');
    },
    ngủ: function () {
        console.log('Zzz… Zzz…');
    }
};
```

```
// Sử dụng các thuộc tính như biến
thông thường
console.log(RongGia.name);
console.log(RongGia.phanLop);
console.log(RongGia.age);


// Sử dụng phương thức như gọi hàm
bình thường
RongGia.bay();
RongGia.phunLửa();
RongGia.ngủ();


// Bạn cũng có thể thêm các thuộc
tính và phương thức
// sau khi đã khởi tạo đối tượng
RongGia.level = 10;
RongGia.chết = function () {
    console.log('Rồng lên đường!');
};
```

Bài tập 8

- Khai báo một đối tượng AnhHung để chứa các thông tin về Anh Hùng của bạn:
  - 1. Các thuộc tính: tên, phân lớp, cấp độ, các chỉ số..
  - 2. Các phương thức ví dụ như đi, chạy, nhảy, ngủ..
- **LEVEL UP!** Chúc mừng bạn lên cấp 8! **XP +800**

Quest 8: Mảng

- Mảng là tập hợp nhiều phần tử, với mỗi phần tử là 1 biến được sắp xếp theo thứ tự và có đánh số chỉ mục (đánh số từ 0) cho từng biến để tiện truy xuất.
  - Mảng được khai báo bên trong cặp ngoặc vuông var tênMảng = [biến1, biến2];.
  - Các biến bên trong cũng được truy xuất bằng ngoặc vuông theo cú pháp tênBiến[sốChỉMục].
  - Số lượng các phần tử bên trong mảng có thể được truy xuất nhanh bằng thuộc tính tênBiến.length
  - Để thêm phần tử vào 1 mảng đã được khởi tạo, bạn cần sử dụng phương thức tênBiến.push()

Ví dụ

```
// Khai báo và khởi tạo mảng ví dụ
var phanlopAnhHung = ['Chiến Binh', 'Sát Thủ', 'Thợ Săn'];
console.log(phanlopAnhHung.length);
console.log(phanlopAnhHung[0]);
console.log(phanlopAnhHung[1]);
console.log(phanlopAnhHung[phanlopAnhHung.length - 1]);
// hàm console.log() rất hữu ích để xem toàn bộ phần tử trong mảng
console.log(phanlopAnhHung);
// Mảng rỗng
var danhHieu = [];


// Thêm phần tử mới
danhHieu.push('Dũng sĩ diệt rồng');
console.log(danhHieu);
phanlopAnhHung.push('Anh Hùng');
console.log(phanlopAnhHung.length);
console.log(phanlopAnhHung[phanlopAnhHung.length - 1]);


// Truy xuất từng phần tử trong mảng với vòng lặp
var soPhanTu = phanlopAnhHung.length;
for (var i = 0; i < soPhanTu; i++) {
    console.log(phanlopAnhHung[i]);
}
```

Bài tập 9

- Hãy sử dụng vòng lặp và hàm prompt() để yêu cầu người dùng nhập vào câu trả lời cho 3 câu hỏi sau:
  - 1. Bạn có người yêu chưa?
  - 2. Bạn có thích ăn rau dền không?
  - 3. Người yêu bạn có thích ăn rau dền không?
- Sau đó, lưu tất cả câu trả lời vào 1 mảng và xuất ngược lại từng câu trả lời ra file HTML.

Bài tập 9

```
var mangCauTraLoi = [];
var mangCauHoi = [];
for(var i = 0; i < 3; i++) {
var cauTraLoi = prompt(mangCauHoi);
mangCauTraLoi.push(cauTraLoi);
}


for(var i = 0; i < 3; i++) {
alert(mangCauTraLoi);
}
```

DOM trong JS

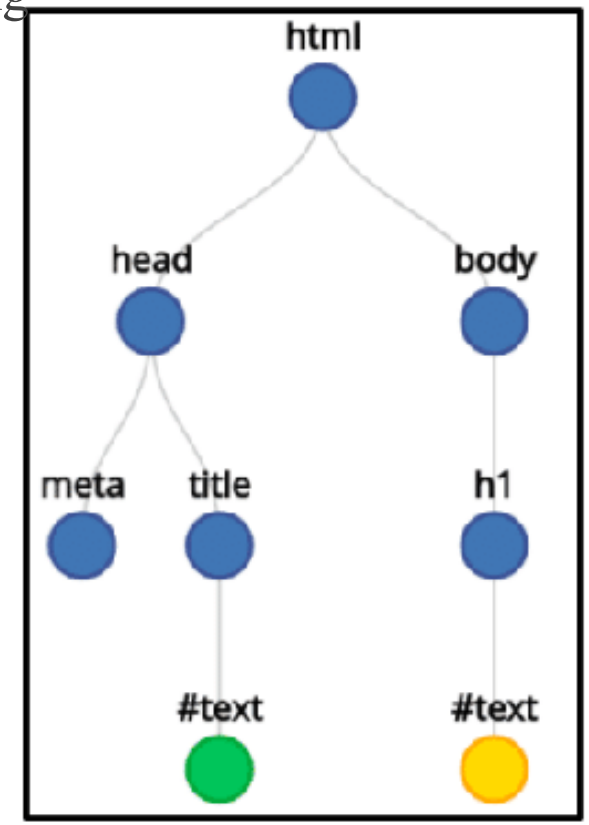
DOM (Document Object Model for HTML)

- **DOM** (Mô hình đối tượng tài liệu) là mô hình được hỗ trợ trong trình duyệt.
- **DOM** định nghĩa chuẩn cách **truy xuất**, **thao tác** các đối tượng trong tài liệu **HTML**.
- Thông qua **DOM**, tất cả các thành phần **HTML**, bao gồm cả **văn bản** và **thuộc tính**, đều có thể truy xuất được.
- **DOM** thể hiện tài liệu **HTML** dưới dạng cấu trúc **cây**, với mỗi **nút** (node) trên cây đại diện cho một phần tử **HTML**.
- Các nút nằm bên trong một nút gọi là **nút con**, còn các nút thuộc cùng mức gọi là **nút anh em**.

DOM (Document Object Model for HTML)

- Công dụng của DOM
  - Định vị các thành phần trong tài liệu theo ID, lớp, tên thẻ (div, p, v.v.) và các đặc điểm khác của chúng
  - Thêm, xóa hoặc sửa đổi các phần tử HTML
  - Sửa đổi bất kỳ thuộc tính nào của thành phần HTML, chẳng hạn như lớp, thuộc tính kiểu và các thuộc tính khác của chúng

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>#text</title>
  </head>
  <body>
    <h1>#text</h1>
  </body>
</html>
```



DOM (Document Object Model for HTML)

- Để tìm kiếm và truy xuất thông tin của một thẻ HTML theo ID:
  - document.**getElementById**('idName');
- Để tìm thẻ html thông qua class:
  - document.**getElementsByClassName**('className');
- Ngoài ra:
  - Returns an HTMLCollection of elements with the given tag name.
    - document.**getElementsByTagName**('tagName');
  - Returns the first element within the document that matches the specified group of selectors.
    - document.**querySelector**(".someclass");
  - Returns a list of the elements within the document (using depth-first pre-order traversal of the document's nodes) that match the specified group of selectors.
    - document.**querySelectorAll**("div.note, div.alert");

DOM (Document Object Model for HTML)

- document.getElementById('hello').innerHTML;
- document.getElementById('hello').innerText;
- const p = document.createElement("p");
  - document.body.appendChild(p);
- element.setAttribute('attrName','attrValue');
- https://github.com/anish000kumar/Javascript-DOM-API-cheatsheet

Event trong JS

DOM event trong javascript

- Xuất ra thông báo '*bạn vừa click*' khi click vào thẻ div sau:
  - Học CSS
- Để thêm sự kiện bằng javascript chúng ta sử dụng cú pháp:
  - element.eventName = function () { //code }
  - Ví dụ:

```
<div id="jav">Học javascript</div>
<script type="text/javascript">
    //chọn thẻ có id = jav
    var element = document.getElementById('jav');
    //Thêm sự kiện
    element.onmouseover = function () {
        alert('Bạn vừa hover chuột lên đối tượng');
    };
</script>
```

DOM event trong javascript

- Hàm addEventListener() có tác dụng thêm sự kiện cho đối tượng HTML, sử dụng với cú pháp:
  - element.addEventListener('eventName',functionName);
  - Ví dụ:

```
<p id="container"> Click</p>
<script type="text/javascript">
    var element = document.getElementById('container');
    element.addEventListener('click', function () {
        alert('Bạn vừa click');
    })
</script>
```

- Hàm removeEventListener() lại có tác dụng là xóa bỏ đi các sự kiện đang được áp lên sự kiện
  - element.removeEventListener('eventName', functionName, option);

Time events trong javascript

- Hàm setTimeout() có tác dụng thực thi hành động sau khoảng thời gian nào đó, và nó chỉ thực hiện đúng một lần:
  - setTimeout(function, milliseconds);
  - Ví dụ:

```
<script type="text/javascript">
    setTimeout(function () {
        alert('Chào mừng bạn đã đến với Khoa CNTTKD');
    }, 3000);
</script>
```

- **clearTimeout()** sẽ có tác dụng hủy hành động của hàm setTimeout()

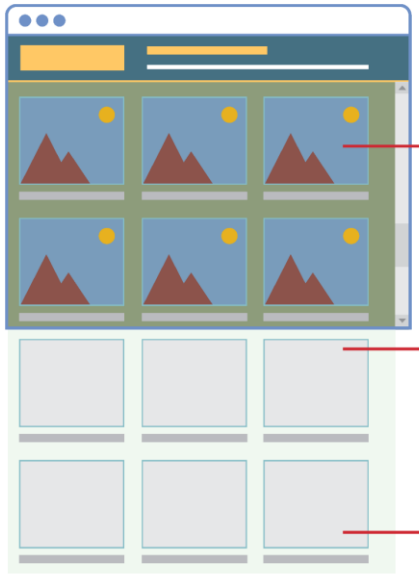
Time events trong javascript

- Hàm **setInterval**() có chức năng là thực hiện hành động sau khoản thời gian bao nhiêu giây, nhưng nó sẽ thực hiện lại nhiều lần như một vòng lặp, cứ sau mỗi khoảng thời gian mà chúng ta xác định nó sẽ lại thực hiện lại:
  - setInterval(function, milliseconds);
  - Ví dụ:

```
<p id="result">0</p>
<script type="text/javascript">
    var result = document.getElementById('result');
    var i = 1;
    setInterval(function () {
        result.innerText = i;
        i++;
    }, 1000);
</script>
```

- **clearIntelval()** sẽ có tác dụng hủy hành động của hàm **setInterval**()

Lazy Loading



1 Images that are not visible will not yet be downloaded. This will improve perceived respon-siveness of the visible portion of the page.

```
<img src="..."
     alt="..." >
```

```
<img data-src="..."
     data-alt="..."
     class="lazy">
```

Since no src attribute is provided, these images are not downloaded by the browser when HTML is received.

```
img.lazy {
   width: 320px;
   height: 240px;
    }
```

CSS for images are sized correctly so that no content shifting occurs when real images are received.

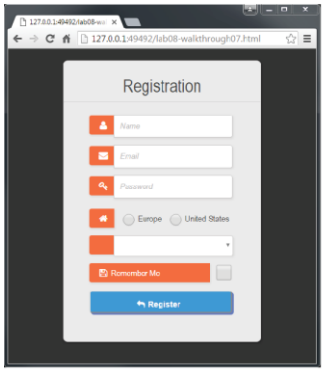
2 Event listeners will be needed for scroll, resize, anc orientationChanged events.

```
document.addEventListener("scroll", lazyload);
window.addEventListener("resize", lazyload);
window.addEventListener("orientationChange", lazyload);
```

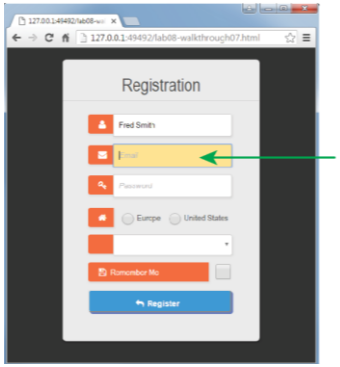
3 For each image, check if now visible. If it is, then change its src attribute to the correct one in data-src. This will make the browser request that file.

```
function lazyLoad() {
   ...
   const images = document.querySelectorAll("img.lazy");
   for (let img of images) {
      if (img.offsetTop < (window.innerHeight + window.pageYOffset))    {
         img.src = img.dataset.src;
         img.alt = img.dataset.alt;
         img.classList.remove('lazy');
      }
   }
```

Responding to form movement events



How form appears when no controls have the focus



When a control has the focus, then chang its background color

```
// This function is going to get called every time the focus or blur events are
// triggered in one of our form's input elements.
function setBackground(e) {
   if (e.type == "focus") {
      e.target.style.backgroundColor = "#FFE393";
   }
   else if (e.type == "blur") {
      e.target.style.backgroundColor = "white";
   }
}

// set up the event listeners only after the DOM is loaded
window.addEventListener("load", function() {
   const cssSelector = "input[type=text],input[type=password]";
   const fields = document.querySelectorAll(cssSelector);
   for (let f of fields) {
      f.addEventListener("focus", setBackground);
      f.addEventListener("blur", setBackground);
   }
});
```

Here we use the `style` property instead of the `classList` property because of specificity conflicts (that is, attribute selectors override class selectors).

Selects the fields that will change.

Assigns the `setBackground()` function to change the background color of the control depending upon whether it has the focus.

Bài tập 10

a) Tạo một hộp Text có tên là Họ và tên, một nút có tên là "Cảnh báo". Khi người dùng click chuột vào nút "Cảnh báo" thì hiện alert với nội dung là "Truy tìm" + nội dung trong Text Họ và tên.

b) Sử dụng file js_menu.html để làm hiệu ứng dropdown menu. Tức là khi người dùng nhấn vào menu thì sẽ mở chi tiết bên trong của nó và đóng các menu con khác

- Gợi ý: sử dụng querySelectorAll, addEventListener() và style.display

Bài tập 10

c) Sử dụng file js_dongho.html để xây dựng đồng hồ đếm ngược. Khi hết giờ sẽ báo đã hết giờ.

- **Gợi ý**: sử dụng getElementById, parseInt, setTimeout và clearTimeout

d) Xây dựng ứng dụng lời yêu thương với giao diện cơ bản như bên dưới (có thể thiết kế lại cho đẹp hơn)

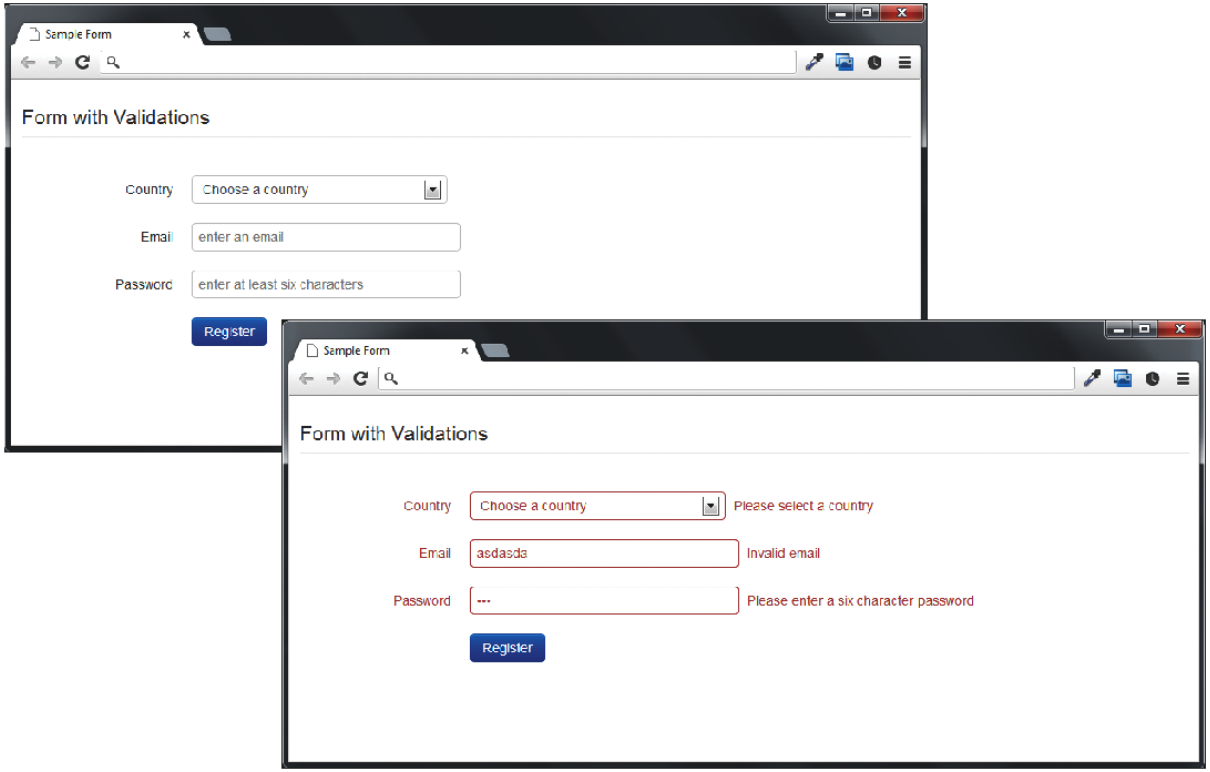
BẠN CÓ THÍCH MÌNH KHÔNG?

CÓ | KHÔNG

- Khi nhấn vào nút KHÔNG, hiện thông báo: có thể Bạn nên chọn lại. Nếu sai 3 lần thì chuyển vị trí của nút KHÔNG và CÓ.
- Khi nhấn vào nút CÓ, hiện thông báo: xin lỗi, bạn không phải là đối tượng của mình.

Bài tập 10

f) Thiết kế màn hình giống hình dưới và hiển thị lỗi khi nhấn vào nút đăng ký:



Bài tập 11 (mỗi câu sẽ có 2 điểm phát biểu)

- **Trò chơi "Ai là triệu phú" đơn giản:** Tạo một trò chơi "Ai là triệu phú" đơn giản, trong đó người chơi phải trả lời các câu hỏi và cố gắng giành được số tiền lớn nhất.
- **Game Tic Tac Toe:** Tạo một phiên bản đơn giản của trò chơi Tic Tac Toe trong không gian 2D, trong đó người chơi cố gắng tạo ra dãy 3 quân cờ giống nhau trên hàng, cột hoặc đường chéo. Sử dụng mảng 2 chiều để biểu diễn bàn cờ.
- **Cho chơi xếp hình**: tạo ngẫu nhiên 8 số vào đổ vào ma trận 3x3 trong đó có 1 ô trống. Người dùng sử dụng các phím mũi tên để di chuyển mảnh trống đến vị trí khác. Hoàn thành khi có hình từ 1->8 theo chiều từ trái qua phải và trên xuống dưới.
- **Game rắn săn mồi**
- **Với những yêu cầu khó có thể nhờ Google hay AI hỗ trợ**

Tài liệu tham khảo

- https://timoday.edu.vn/
- https://webcoban.vn/html/tai-lieu-hoc-html
- Fundamentals of Web Development 3nd Edition
- javascript cơ bản – hành trình của một anh hùng
- 30 Days Of JavaScript

UEH UNIVERSITY

Q&A